

SHAP-Guided, Probabilistic, and Hierarchical Sufficient Input Subsets for Efficient Vision Interpretability

Yingjie Huang

Department of Computer Science, University of California, Los Angeles

yingjieh512@g.ucla.edu

Abstract

Sufficient Input Subsets (SIS) explain a black-box prediction by identifying minimal observed feature sets that keep the model output above a task-specific confidence threshold. This report extends Google Research's NumPy SIS implementation with three research-engineering additions: a SHAP-inspired acceleration layer, a probabilistic sampler for explanation uncertainty, and a hierarchical coarse-to-fine vision explanation procedure. The implementation preserves the original `sis_collection` result format where practical, adds stability metrics for robustness analysis, and includes laptop-scale benchmarks. On a deterministic 8 by 8 synthetic image benchmark, the SHAP-guided method reduced individual model evaluations from 3974 to 142 while preserving subset size 7 and final sufficiency score 0.9997. This corresponds to a measured 96.43% reduction in individual model evaluations on this benchmark. The result should be interpreted as a measured benchmark outcome, not a universal guarantee.

1 Introduction

Modern vision models can produce high-confidence predictions without exposing which input evidence was necessary for the decision. Sufficient Input Subsets address this by searching for a sparse subset of input features whose observed values alone are enough for the same high-confidence decision [1]. For a black-box scoring function `f`, input `x`, mask baseline `x_masked`, and threshold `tau`, SIS seeks a mask `m` such that `f(x_m) >= tau`, where masked-out positions are replaced by the fully masked input. The original Google Research implementation provides a clean NumPy reference implementation with `sis_collection`, `find_sis`, and the `SISResult` container [2].

The main practical limitation is computational cost. Vanilla SIS uses iterative backward selection. At each stage it evaluates many candidate masked variants, which is faithful but expensive for image inputs with many pixels or regions. This project asks whether a lightweight importance pass can reduce avoidable evaluations while preserving the sufficiency contract, and whether repeated and hierarchical variants can provide more informative explanations than a single deterministic mask.

The contributions are:

- A SHAP-inspired SIS wrapper that scores features or feature groups with batched perturbations, memoizes masked evaluations, and returns SIS-compatible collections with diagnostics.
- A Probabilistic SIS procedure that samples noisy plausible explanations and estimates per-feature inclusion probabilities.
- A Hierarchical SIS procedure that moves from coarse grid regions to fine pixel-level explanations.
- Stability metrics that quantify mask overlap, subset-size variance, confidence retention, and explanation drift under perturbations.
- A reproducible benchmark and demo that report real runtime, model-call counts, individual evaluations, subset size, sufficiency score, and stability.

2 Related Work

SIS was introduced by Carter, Mueller, Jain, and Gifford as a black-box interpretability method for finding sparse sufficient rationales [1]. The implementation used here builds directly on the Google Research `sufficient_input_subsets` code rather than replacing it [2]. SHAP frames feature attribution through Shapley values and provides a unifying view of additive feature explanations [3]. The acceleration in this project is SHAP-inspired: when the optional `shap` package is unavailable, the implementation uses a perturbation-based confidence-drop estimator rather than exact Shapley values. The vision demo uses the scikit-learn ecosystem and

its lightweight digits tooling when available [4]. Stability and adversarial sensitivity are motivated by the broader observation that small input perturbations can expose brittle model behavior [5].

3 Methodology

3.1 Baseline SIS

The original SIS API is preserved. `sis_collection(f_batch, threshold, initial_input, fully_masked_input, initial_mask=None)` returns a list of `SISResult` objects. Each `SISResult` contains `sis`, `ordering_over_entire_backselect`, `values_over_entire_backselect`, and `mask`. Masks use the original convention: `True` means the feature is present and `False` means it is masked. Broadcastable masks remain supported, so a user can mask individual pixels, rows, columns, channels, or regions.

3.2 SHAP-Guided SIS

The extension adds `shap_guided_sis_collection`. It first estimates importance for each feature or feature group. In the default perturbation mode, it evaluates the confidence drop caused by masking one group at a time:

```
importance(g) = f(x_current) - f(x_current with group g masked)
```

Feature scoring is batched through `f_batch`, and masked inputs are memoized by boolean mask bytes. After ranking groups by importance, the algorithm constructs a sufficient subset by adding high-ranked groups until the threshold is reached, then prunes removable groups while preserving `f(x_m) >= tau`. When `return_diagnostics=True`, the wrapper reports runtime, batched calls, individual evaluations, selected feature count, sufficiency score, and estimated speedup if a baseline is provided.

3.3 Probabilistic SIS

Probabilistic SIS models explanation uncertainty by repeated stochastic sampling. Each sample perturbs the ranking scores with reproducible Gaussian noise controlled by `noise_scale` and `random_state`. The output is a list of sampled SIS collections plus an inclusion probability map:

```
P_i = number of sampled masks including feature i / number of samples
```

The method also reports mean subset size, subset-size variance, threshold-met rate, and pairwise explanation stability.

3.4 Hierarchical SIS

Hierarchical SIS provides multi-scale explanations. The implementation supports grid and pixel modes without heavy dependencies. At level `l`, the image is partitioned into grid cells, SIS is run over those regions, and the selected union mask becomes the active search space for the next finer level. In pixel mode, the final level uses singleton pixel groups. The returned object includes a tree of level diagnostics, masks at each level, final masks, model-call counts, and fallback notes.

3.5 Stability Metrics

The stability module provides mask IoU, mask F1, pairwise explanation stability, perturbation stability, and adversarial sensitivity. Given masks `A` and `B`, IoU is `|A intersection B| / |A union B|`; F1 is `2TP / (2TP + FP + FN)`. Perturbation stability repeatedly applies a user-provided perturbation function, recomputes explanations, and reports explanation drift and confidence retention. These metrics do not prove adversarial robustness, but they establish a measurable baseline for identifying brittle explanations.

4 Experiments

4.1 Implementation and Test Status

All experiments were run locally from `C:\Users\Yingjie Huang\Downloads\sis-research-work`. The extension test suite passed 14 out of 14 tests, and the original Google SIS test suite passed 18 out of 18 tests through `python -m sufficient\_input\_subsets.sis\_test`. The benchmark command was `python -m sufficient\_input\_subsets.benchmark\_sis`. The demo command was `python -m sufficient\_input\_subsets.vision\_demo --n\_probabilistic\_samples 3`.

4.2 Synthetic Image Benchmark

The benchmark uses a deterministic synthetic 8 by 8 image classifier with initial confidence 1.0000, fully masked confidence 0.0180, and threshold 0.75. This setup is small enough for a laptop but still exposes the computational pattern of SIS search.

Method	Runtime (s)	f_batch calls	Individual evals	Subset size	Score	Met threshold	Speedup vs baseline	Stability
original_sis	0.0074	128	3974	7	0.9997	yes	0.00%	0.833
shap_guided_sis	0.0020	19	142	7	0.9997	yes	96.43%	0.750
probabilistic_sis	0.0109	100	715	8	0.9999	yes	82.01%	0.833
hierarchical_sis	0.0035	46	129	7	0.9997	yes	96.75%	0.750

SHAP-guided SIS achieved a measured 96.43% reduction in individual model evaluations relative to original SIS on this benchmark. The speedup was computed as `(baseline evaluations - method evaluations) / baseline evaluations`. The measured reduction exceeds the target of approximately 20 percent on this benchmark, but the paper reports the actual value rather than assuming it will transfer unchanged to larger models.

4.3 Vision Demo

The runnable demo selected the `sklearn\_digits` path, target class 7, initial confidence 0.9996, and threshold 0.8497. Baseline SIS and SHAP-guided SIS both selected subset size 8 and 8, respectively. Probabilistic SIS reported mean subset size 8.00, and Hierarchical SIS produced a final subset size of 3. Under small perturbations, the demo measured mean explanation drift 0.0222 and mean confidence retention 0.9999. The demo also saved visual artifacts for the original image, baseline SIS mask, SHAP-guided SIS mask, probabilistic heatmap, hierarchical regions, and perturbation stability.

5 Analysis

The synthetic benchmark shows that ranking features before SIS search can drastically reduce the number of evaluated masked variants while preserving sufficiency. The SHAP-guided method matched the baseline final sufficiency score of 0.9997 and subset size 7, but used 142 individual evaluations instead of 3974. Hierarchical SIS also reduced evaluations because it searched over coarse regions before pixel-level refinement. Probabilistic SIS used more evaluations than the single SHAP-guided run because it intentionally samples multiple explanations, but it still used far fewer evaluations than original SIS in this benchmark.

The stability results should be interpreted carefully. Original SIS and probabilistic SIS had stability score 0.833 on the synthetic perturbation probe, while SHAP-guided and hierarchical SIS had 0.750. This suggests a tradeoff: acceleration and hierarchy can reduce computation but may slightly alter mask stability under the tested perturbations. The probabilistic inclusion map helps expose this uncertainty instead of hiding it behind one deterministic explanation.

The project does not claim universal 20 percent overhead reduction. The benchmark demonstrates a measured reduction on a controlled task. Larger images, deep neural networks, different masking baselines, and different thresholds may change the result. The code includes optimization hooks such as feature groups, batching, caching, candidate limits, and hierarchy levels so the same measurement protocol can be repeated honestly on new workloads.

6 Conclusion

This work turns the original SIS implementation into a broader research-engineering framework for efficient and robust interpretability. It preserves the core SIS sufficiency semantics, adds a SHAP-inspired acceleration layer, samples probabilistic

explanations, refines explanations hierarchically across scales, and measures stability under perturbations. The current laptop-scale experiments validate correctness through tests and show substantial measured evaluation reduction on a synthetic benchmark. Future work should evaluate the framework on larger vision models, compare exact SHAP and gradient-based guidance, add stronger superpixel and semantic-region hierarchies, and test adversarial attacks such as FGSM or PGD.

Artifact Consistency Statement

The Markdown and PDF versions of this paper are generated from the same canonical source payload by ``tools/generate_neurips_paper.py``. No separate prose edits are applied to the PDF. The payload SHA256 appears below and is identical for both artifacts.

CANONICAL_PAYLOAD_SHA256: 4558e412eef9dc584aae9fd245b295fef05d84fb660906a72090db586c607b8b

References

- [1] Brandon Carter, Jonas Mueller, Siddhartha Jain, and David K. Gifford. What made you do this? Understanding black-box decisions with sufficient input subsets. arXiv:1810.03805, 2018. <https://arxiv.org/abs/1810.03805>
- [2] Google Research Authors. sufficient_input_subsets implementation in google-research. https://github.com/google-research/google-research/tree/master/sufficient_input_subsets
- [3] Scott M. Lundberg and Su-In Lee. A Unified Approach to Interpreting Model Predictions. Advances in Neural Information Processing Systems 30, 2017. <https://proceedings.neurips.cc/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html>
- [4] Fabian Pedregosa et al. Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12:2825-2830, 2011. <https://jmlr.org/papers/v12/pedregosa11a.html>
- [5] Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and Harnessing Adversarial Examples. arXiv:1412.6572, 2014. <https://arxiv.org/abs/1412.6572>